Task 1: Requirements (IoT Context)

Prompt

Generate user requirements for an IoT indoor light sensor application that monitors two rooms (living room mapped to sensor-1/esp32_01, bedroom mapped to sensor-2/esp32_02) using BH1750 sensors mounted on ESP32 boards. The ESP32 POSTs JSON payloads {sensor_id, timestamp, lux} every 5 minutes to a Flask backend deployed on Render. Data is stored in MongoDB Atlas collections: event (time-series, 90-day TTL), room_state (current lux per room), daily_usage (aggregated on/off seconds), and alert (SUDDEN_LUX_DROP, LIGHT_STUCK_ON). The real-time dashboard shows sensor badges and an hourly lux graph. Include functional and non-functional requirements.

Result Summary

Copilot produced a structured list of functional requirements — for example, "The system shall accept lux readings via POST /api/v1/sensors/data and persist them to the event collection within 500ms" — along with non-functional ones like dashboard refresh timing and MongoDB uptime targets. It also generated user-facing requirements around the hourly graph visibility. Sensor-to-room mapping was correctly identified as a system constraint, and alert generation was flagged as a derived requirement from sensor data rather than a direct user input.

Pro
Copilot correctly separated the physical layer (ESP32 POST interval) from the application layer (dashboard polling), producing requirements that trace directly to real components in the codebase — including SENSOR_ROOM_MAP, LUX_LIGHTS_ON_THRESHOLD = 25.0, and the alert collection schema.

Con
It never tied acceptance criteria to the actual lux thresholds from the code. It wrote things like "the system shall classify light levels" without specifying the exact boundaries (Dark < 15, Dim < 25, Normal < 35, Bright < 50) that are hardcoded in get_sensor_status(). A reviewer reading only the AI output couldn't write a passing test without digging into the source.

Task 2: Architecture (IoT Pipeline)

Prompt

Design the system architecture for an IoT light sensor pipeline with these exact components: (1) ESP32 boards running BH1750_sensor.ino POST JSON {sensor_id, timestamp, lux} to /api/v1/sensors/data every 5 minutes using HTTPS with client.setInsecure(); (2) Flask backend on Render with SENSOR_ROOM_MAP {sensor-1:living, sensor-2:bedroom, esp32_01:living, esp32_02:bedroom}; (3) MongoDB Atlas with collections: event, room_state, daily_usage, sensor_hourly, alert, device; (4) Dashboard frontend polling /api/v1/sensors/badges and

/api/v1/sensors/hourly_graph. Describe each component, its responsibility, and the data flow between them. Include where lux classification (LUX_LIGHTS_ON_THRESHOLD = 25.0) and alert generation occur.

Result Summary

Copilot produced a layered architecture description covering: Embedded Layer (ESP32 + BH1750, HTTPS POST to Render), API Layer (Flask routes for sensor ingestion, badge aggregation, hourly graph), Data Layer (MongoDB Atlas with event as append-only source of truth and room_state as the fast-read cache), and Presentation Layer (dashboard HTML polling the two API endpoints). It correctly placed lux classification logic in the API layer and described the upsert pattern for room_state as a state derived from the event collection.

Pro
The architecture correctly identified the dual-collection pattern — event for immutable append and room_state for current-state caching — which matches the actual MongoDB schema, including the 90-day TTL index on the event collection and the upsert-on-each-event behavior of room_state.

Con
Copilot suggested adding an MQTT broker between the ESP32 and Flask as a "best practice" for IoT systems. While that's technically reasonable in general, it directly contradicts the design decision documented in 001-event-driven-architecture.md and would require rewriting the embedded firmware. It introduced an unsolicited architectural change without acknowledging the existing direct-HTTPS design.

Task 3: Implementation (Backend API)

Prompt

Write a Python Flask API endpoint for POST /api/v1/sensors/data that:

1. Accepts JSON body: {sensor_id: str, timestamp: str ISO-8601, lux: float}
2. Validates lux is a number in range 0 to 65535; returns 400 if invalid
3. Maps sensor_id to room using: {sensor-1: living, sensor-2: bedroom, esp32_01: living, esp32_02: bedroom}; returns 400 if sensor_id unknown
4. Sets light_state to "ON" if lux >= 25.0, else "OFF"
5. Inserts a document into MongoDB collection named "event" with fields: event_id (string), ts (datetime), room_id (string), device_id (same as sensor_id), light_state (string), lux (float)
6. Upserts MongoDB collection "room_state" with latest lux, light_state, and updated_at timestamp for the room
7. Returns 201 with {success: true, room_id, light_state, lux} on success

Result Summary

Copilot generated a complete Flask route with input validation using request.get_json(), a try/except block around both MongoDB operations, proper use of $set with upsert=True for the room_state collection, and a generated event_id using uuid4(). The light_state logic correctly used >= 25.0 as the threshold. The 400 validation responses included specific error messages for missing fields, out-of-range lux, and unknown sensor IDs.

Pro

The generated endpoint matches the actual structure already in app.py — it used request.get_json(silent=True), matched the SENSOR_ROOM_MAP dictionary pattern, and correctly separated the event insert from the room_state upsert into two sequential MongoDB operations, consistent with how the existing /api/v1/sensors/data route is structured in the codebase.

Con

Copilot didn't include the sensor_hourly collection update that the existing endpoint also writes to (used by the /api/v1/sensors/hourly_graph route). The generated code would pass basic integration tests but would silently break the hourly graph on the dashboard because the hourly aggregation bucket would never be written. This omission isn't detectable from the prompt alone — it requires knowing the full system behavior.

Task 4: Testing (Sensor Data Logic)

Prompt

Generate pytest unit tests for the get_sensor_status(lux) function in dashboard/app.py. The function accepts a float lux value and returns a dict with keys: level (str), color (str), icon (str). The classification thresholds are:

- lux < 15 → level = "Dark", color = "#1a1a2e", icon = "🌑"
- lux < 25 → level = "Dim", color = "#16213e", icon = "🌒"
- lux < 35 → level = "Normal", color = "#e94560", icon = "🌕"
- lux < 50 → level = "Bright", color = "#f39c12", icon = "🌟"
- lux >= 50 → level = "Very Bright", color = "#f1c40f", icon = "☀️"

Include boundary tests at exact threshold values (14.9, 15.0, 24.9, 25.0, 34.9, 35.0, 49.9, 50.0), edge cases (0, -1, 65535), and verify all three dict keys are present in every result.

Result Summary

Copilot produced a pytest file with 14 test functions organized around boundary and edge cases. Each test called get_sensor_status() directly and asserted the exact level string, color hex code, and icon emoji. It used pytest.mark.parametrize for the boundary group, checking both sides of each threshold (e.g., 14.9 → Dark, 15.0 → Dim). The edge case tests included lux=0 (Dark),

lux=-1 (Dark, since -1 < 15), and lux=65535 (Very Bright).

Pro

The parametrized boundary tests directly match the exact threshold values in the source code and would catch any off-by-one error if a future developer changed < 15 to <= 15. These tests extend the existing tests/app_test.py, which only has three tests (test_sensor_status_dark, test_sensor_status_normal, test_generate_sensor_range) and doesn't cover any boundary conditions.

Con

Copilot didn't flag that lux = -1 is a physically impossible sensor reading that the API layer should reject before it ever reaches get_sensor_status(). The test asserts it returns "Dark" without raising an error — implying the function handles invalid input silently. This could give false confidence that the system gracefully handles bad sensor data when, in fact, the validation gap is in the upstream POST endpoint, not in this function.

Task 5: Verification (System Behavior)

Prompt

Verify end-to-end system behavior for this scenario in the IoT light sensor application:
At 2:00 PM, Room A (living room, sensor esp32_01) sends lux=480. At 2:05 PM, the same sensor sends lux=3.
Verify the following system-level behaviors:

1. Both POST requests to /api/v1/sensors/data return HTTP 201.
2. MongoDB event collection contains two documents for device_id=esp32_01: first with lux=480 and light_state=ON, second with lux=3 and light_state=OFF (since LUX_LIGHTS_ON_THRESHOLD=25.0).
3. MongoDB room_state for room_id=living is updated to lux=3, light_state=OFF after the second POST.
4. MongoDB alert collection receives a new document of type SUDDEN_LUX_DROP for room_id=living.
5. Dashboard /api/v1/sensors/badges returns sensor1 with light_state=OFF and lux=3 after the second POST.

Write verification steps as pytest integration tests using a real MongoDB test database and the Flask test client.

Result Summary

Copilot generated a pytest integration test module using Flask's app.test_client() and a mongomock fixture to simulate MongoDB Atlas. It produced five test functions corresponding to

the five verification checks: HTTP response codes, event collection document assertions, room_state upsert verification, alert insertion check, and badge API response validation. Each test used mongomock.patch() to intercept MongoDB calls and assert response.status_code == 201 for the POST checks.

Pro

The test for behavior #3 (room_state upsert) correctly asserted that only one document exists in room_state for room_id=living after both POSTs — verifying the upsert semantics rather than a simple insert — which matches the actual updateOne(..., upsert=True) logic in the backend and would catch a regression if that was accidentally changed to insert_one.

Con

The alert generation test (behavior #4) will always fail against the real system because the actual app.py does not yet implement automatic SUDDEN_LUX_DROP alert generation in the POST endpoint — alerts in the current codebase are written by a separate process or manual trigger. Copilot assumed this feature existed because it appeared in the MongoDB schema in database-schema.md, but the schema documents a planned state, not the current implemented state. This creates a false-failing test that misleads a developer into thinking the feature is broken rather than unimplemented.